

# Segmentation fault on SVM LIB #18891

Closed

#21336

pabloec20 opened on Nov 21, 2020 · edited by NicolasHug

Edits ...

## Description:

In Scikit-learn version 0.23.2 calling the `predict()` method maliciously crafted model SVM can result in a segmentation fault. Such models can be introduced via pickle, json, or any other model permanence standard. The behaviour is triggered when one of the members of the `_n_support` array has a very large value, example `1000000` when calling `libsvm.predict()`

##### Tested environment:

Ubuntu 9.3.0-17ubuntu1~20.04

Python 3.8.5 (default, Jul 28 2020, 12:59:40)

[GCC 9.3.0] on linux

Numpy version: '1.19.2'

Sklearn.version: '0.23.2'

## Steps/Code to Reproduce

```
from sklearn import svm
from sklearn import datasets

if __name__ == '__main__':
    X,y = datasets.load_iris(return_X_y=True)
    clf = svm.SVC()
    clf.fit(X, y)
    clf._n_support[0] = 1000000
    y_pred = clf.predict(X)
```

## Expected Results

not to fail

## Actual Results

Segmentation fault, this is a debugger trace

```
Thread 1 "python3-dbg" received signal SIGSEGV, Segmentation fault. 0x00007ffffd7174df7 in
svm_predict_values () from /home/pablo/.local/lib/python3.8/site-
packages/sklearn/svm/_libsvm.cpython-38-x86_64-linux-gnu.so (gdb) bt #0 0x00007ffffd7174df7
in svm_predict_values () from /home/pablo/.local/lib/python3.8/site-
packages/sklearn/svm/_libsvm.cpython-38-x86_64-linux-gnu.so #1 0x00007ffffd717504f in
svm_predict () from /home/pablo/.local/lib/python3.8/site-
packages/sklearn/svm/_libsvm.cpython-38-x86_64-linux-gnu.so #2 0x00007ffffd7163e57 in
copy_predict () from /home/pablo/.local/lib/python3.8/site-
packages/sklearn/svm/_libsvm.cpython-38-x86_64-linux-gnu.so #3 0x00007ffffd716bb3a in ?? ()
from /home/pablo/.local/lib/python3.8/site-packages/sklearn/svm/_libsvm.cpython-38-x86_64-
linux-gnu.so #4 0x00007ffffd716d47d in ?? () from /home/pablo/.local/lib/python3.8/site-
packages/sklearn/svm/_libsvm.cpython-38-x86_64-linux-gnu.so #5 0x000000000043663e in
cfunction_call_varargs (func=0x7ffffd76e9dd0, args=0x7ffffd40fc1d0, kwargs=0x7ffffd40fbdd0) at
../Objects/call.c:742 #6 0x000000000043920c in PyCFunction_Call (func=<optimized out>, args=
<optimized out>, kwargs=<optimized out>) at ../Objects/call.c:772 #7 0x000000000043708b in
_PyObject_MakeTpCall (callable=callable@entry=0x7ffffd76e9dd0, args=args@entry=0x18433a0,
nargs=<optimized out>, keywords=keywords@entry=0x7ffffd7821de0) at ../Objects/call.c:159 #8
0x000000000044ebe5b in _PyObject_Vectorcall (kwnames=0x7ffffd7821de0,
nargsf=9223372036854775816, args=0x18433a0, callable=0x7ffffd76e9dd0) at
../Include/cpython/abstract.h:125 #9 call_function (kwnames=0x7ffffd7821de0, oparg=<optimized
out>, pp_stack=<synthetic pointer>, tstate=0x954500) at ../Python/ceval.c:4963 #10
_PyEval_EvalFrameDefault (f=0x1843210, throwflag=<optimized out>) at ../Python/ceval.c:3515
#11 0x000000000044df2ef in PyEval_EvalFrameEx (f=
```

## Versions

System:

python: 3.8.5 (default, Jul 28 2020, 12:59:40) [GCC 9.3.0]

executable: /usr/bin/python3

machine: Linux-5.4.0-53-generic-x86\_64-with-glibc2.29

Python dependencies:

pip: 20.0.2

setuptools: 45.2.0

sklearn: 0.23.2

numpy: 1.19.2

scipy: 1.5.2

Cython: None

pandas: 1.1.2

matplotlib: 3.3.2

joblib: 0.16.0

threadpoolctl: 2.1.0

Built with OpenMP: True



 **pabloec20** added **Bug: triage** on Nov 21, 2020

 carnil on Nov 22, 2020

...

This issue was assigned [CVE-2020-28975](#)

NicolasHug on Nov 22, 2020

Member ...

```
clf._n_support[0] = 1000000
```

It's expected that manually changing a private attribute will lead to wrong results, or even segfault as is the case here.

May I ask why you want to change the number of SVs, and why you expected it to work? Changing it likely leads the program to overshoot indexing in an array somewhere. For ref, there are 7 SVs for class 0.

  **NicolasHug** closed this as completed on Nov 22, 2020

pabloec20 on Nov 22, 2020

Author ...

It should not segfault but fail gracefully.

NicolasHug on Nov 23, 2020

Member ...

The snippet above is changing a *private* attribute (it has a leading underscore).

By definition, these aren't supposed to be changed by the user. When they're used by the estimator, it's fair to assume that they are properly set to a correct value.

We can't safe-guard against every single misuse, and this is a misuse.

pabloec20 on Nov 23, 2020

Author ...

Consider the common scenario when a model is trained in a research machine and then transfered for production use to live environment, if a malicious actor is able to take a hold of the model and change it, it should not be able to trigger a segmentation fault, please bear in mind the segfaults may have security implications other than crashing the process.

NicolasHug on Nov 23, 2020

Member ...

if a malicious actor

This is where it's out of scope here: we can't guard against everything. We have a responsibility to provide safe code when that code is used under the limits of what's a normal use-case, but that's pretty much it. Private attributes shouldn't be modified, and it's up to users to make sure that the estimator isn't maliciously altered.

I might go on a limb and use a poor analogy but when I buy a car, I can't complain that it breaks if I replace the steering wheel by a potato.

If you really want to prevent segfaults, maybe you can intercept the `SIGSEGV` ?

pabloec20 on Nov 27, 2020 · edited by pabloec20

Edits ▾

Author

...

To reuse your example: If someone switches the steering wheel for a potato, the car should not completely break/fail unrecoverably the moment you turn it on and if it happens it would be considered a security risk would it not? You would not just say the driver to wear a helmet instead or would you?

jnothman on Nov 27, 2020 via email

Member

...

Pickles are unsafe in any case. If you submit a pull request to avoid the segfault we can consider it.

1

pabloec20 on Dec 1, 2020

Author

...

There are other ways to export a model which are not via pickle, and this does not require pickle at all.

bkreider on Oct 15, 2021

...

This is where it's out of scope here: we can't guard against everything.

Isn't the model loader improperly validating the model and then segfaulting? Is pickle the only way to load a model?

The proof of concept exploit code is just using the private attribute to shorten the example. Perhaps it should have used a comment there describing "loading a model with a modified n value"?

This high CVE score is tripping up people trying to use scikit learn in regulated and secure environments.

1

rth on Oct 15, 2021

Member

...

This high CVE score is tripping up people trying to use scikit learn in regulated and secure environments.

Hmm, yes <https://nvd.nist.gov/vuln/detail/CVE-2020-28975> and those are really used for enterprise deployment. We could do a bit more sanity checks in

[scikit-learn/sklearn/svm/\\_libsvm.pyx](#)

Line 279 in [b43f057](#)

```
279     def predict(np.ndarray[np.float64_t, ndim=2, mode='c'] X,
```

to make that CVE go away, even if it would be difficult to be fully fail proof. A PR would be welcome.

1



 **thomasjpfan** reopened this on Oct 15, 2021



 **thomasjpfan** mentioned this on Oct 15, 2021



FIX Prevents segfault in SVC when internals are altered #21336



 **glemaitre** closed this as completed in [#21336](#) on Oct 20, 2021

Sign up for free

to join this conversation on GitHub. Already have an account? [Sign in to comment](#)

#### Assignees

No one assigned

#### Labels

No labels

#### Type

No type

#### Projects

No projects

#### Milestone

No milestone

#### Relationships

None yet

#### Development



Code with agent mode



**FIX Prevents segfault in SVC when internals are altered**

scikit-learn/scikit-learn

#### Participants



+2

